

Subset Sum Divisible by K

Problem Statement

Given an array `arr[]` of positive integers and an integer `k`, determine whether there exists a **non-empty subset** whose sum is divisible by `k`.

Return:

- `True` → if such a subset exists.
- `False` → otherwise.

Examples

Example 1

```
arr = [3, 1, 7, 5]
k = 6
```

Subset:

```
[1, 5]
```

Sum:

```
1 + 5 = 6
6 % 6 = 0
```

Answer:

```
True
```

Example 2

```
arr = [1, 2, 4]
k = 8
```

No subset sum is divisible by 8.

Answer:

False

Step 1: Brute Force Thinking

Whenever you hear:

Subset

Your first thought should be:

Take
Not Take

Recursion:

`f(i, sum)`

Meaning:

Starting from index `i`, can I form a subset whose current sum is `sum` and eventually becomes divisible by `k`?

Recurrence:

```
take = f(i+1, sum + arr[i])  
notTake = f(i+1, sum)  
  
return take or notTake
```

Complexity:

```
Time : O(2^n)  
Space: O(n)
```

Main Observation

Do we really need the entire `sum`?

No.

The problem only asks:

```
sum % k == 0
```

So:

```
sum = 17  
17 % 5 = 2
```

and

```
sum = 2  
2 % 5 = 2
```

For this problem, both states are identical.

Hence we don't need:

```
sum
```

We only need:

```
sum % k
```

State Compression

Instead of:

```
f(i, sum)
```

we use:

```
f(i, rem)
```

where

```
rem = sum % k
```

Why is this useful?

Possible values of remainder are only:

```
0, 1, 2, ..., k-1
```

Only k possibilities!

This reduces the state space drastically.

Building the Recurrence

Suppose:

```
current remainder = rem
```

At index i , we have two choices.

Choice 1: Don't Take

Sum doesn't change.

Remainder also doesn't change.

```
notTake = f(i+1, rem)
```

Choice 2: Take

New sum:

```
sum + arr[i]
```

New remainder:

```
(sum + arr[i]) % k
```

Since:

```
sum % k = rem
```

we can write:

```
newRem = (rem + arr[i]) % k
```

Therefore:

```
take = f(i+1, newRem)
```

Final Recurrence

```
f(i, rem):  
    take = f(i+1, (rem + arr[i]) % k)  
    notTake = f(i+1, rem)  
    return take or notTake
```

Handling Non-Empty Subset

The problem specifically says:

```
Non-empty subset
```

If we start with:

```
f(0,0)
```

then the empty subset also has remainder 0.

We must avoid accepting it.

Solution

Add one more state:

```
taken
```

Meaning:

```
taken = 0 → no element selected yet  
taken = 1 → at least one element selected
```

State becomes:

```
f(i, rem, taken)
```

Base Case

```
if i == n:  
    return rem == 0 and taken
```

We only return `True` if:

1. remainder is 0
 2. subset is non-empty.
-

Final Recurrence

```
take = f(  
    i+1,  
    (rem + arr[i]) % k,  
    1  
)  
  
notTake = f(  
    i+1,  
    rem,  
    taken  
)  
  
return take or notTake
```

Memoization

DP State

```
dp[i][rem][taken]
```

Meaning:

Starting from index `i`, with current remainder `rem`, and whether we have already selected an element, can we form a valid subset?

Dimensions

Index

```
0 → n
```

Remainder

```
0 → k-1
```

Taken

```
0 or 1
```

Therefore:

```
DP size = (n+1) × k × 2
```

Memoization Code

```
dp = [[[-1]*2 for _ in range(k)] for _ in range(n+1)]

def f(i, rem, taken):

    if i == n:
        return rem == 0 and taken

    if dp[i][rem][taken] != -1:
        return dp[i][rem][taken]

    take = f(
        i+1,
        (rem + arr[i]) % k,
        1
    )

    notTake = f(
        i+1,
        rem,
        taken
    )

    dp[i][rem][taken] = take or notTake
    return dp[i][rem][taken]
```

Answer:

```
f(0,0,0)
```

Tabulation

DP Table

```
dp[i][rem][taken]
```

Base Case

```
for rem in range(k):
    dp[n][rem][0] = False
    dp[n][rem][1] = (rem == 0)
```

Transition

```
notTake = dp[i+1][rem][taken]

newRem = (rem + arr[i]) % k
take = dp[i+1][newRem][1]

dp[i][rem][taken] = take or notTake
```

Tabulation Code

```
def subsetDivisible(arr, k):
    n = len(arr)

    dp = [[[False]*2 for _ in range(k)]
           for _ in range(n+1)]

    for rem in range(k):
        dp[n][rem][0] = False
        dp[n][rem][1] = (rem == 0)

    for i in range(n-1, -1, -1):
        for rem in range(k):
            for taken in range(2):

                notTake = dp[i+1][rem][taken]

                newRem = (rem + arr[i]) % k
                take = dp[i+1][newRem][1]

                dp[i][rem][taken] = (
                    take or notTake
                )
```

```
return dp[0][0][0]
```

Complexity:

```
Time   : O(n*k)
Space  : O(n*k)
```

Space Optimized Idea (Best Solution)

Instead of storing all states, store only:

```
All possible remainders that can be formed.
```

Observation

Suppose:

```
possible = {2,4}
```

This means:

```
There exists some subset with remainder 2.
There exists some subset with remainder 4.
```

Now next number is:

```
num = 7
k = 5
```

We have two choices.

Start a New Subset

$$7 \% 5 = 2$$

Add:

$$2$$

Extend Old Subsets

From remainder 2:

$$(2+7)\%5 = 4$$

From remainder 4:

$$(4+7)\%5 = 1$$

General Transition

For every remainder:

$$\text{newRem} = (\text{oldRem} + \text{num}) \% k$$

Algorithm

For every number:

1. Start a new subset.
2. Extend all previous subsets.
3. Update possible remainders.

Dry Run

```
arr = [2,4]
k = 3
```

Initially:

```
possible = {}
```

Process 2

```
new = {}

2 % 3 = 2

possible = {2}
```

Process 4

Start new subset:

```
4 % 3 = 1
```

```
new = {2,1}
```

Extend old remainder:

```
(2+4)%3 = 0
```

```
new = {2,1,0}
```

Since:

```
0 in possible
```

Answer:

True

Subset:

[2,4]

Space Optimized Code

```
def subsetDivisible(arr, k):  
    possible = set()  
  
    for num in arr:  
        new = possible.copy()  
  
        # subset containing only num  
        new.add(num % k)  
  
        # append num to old subsets  
        for rem in possible:  
            new.add((rem + num) % k)  
  
        possible = new  
  
        if 0 in possible:  
            return True  
  
    return False
```

Complexity

Time : $O(n*k)$
Space : $O(k)$

Because there can be at most k remainders.

Pigeonhole Principle Optimization

If:

$n > k$

Answer is always:

True

Reason:

There are only:

k possible remainders

but we have more than k prefix sums.

Hence:

1. Some prefix remainder is 0, OR
2. Two prefix sums have same remainder.

In both cases, some subset/subarray sum is divisible by k .

Final Optimized Solution

```
def subsetDivisible(arr, k):  
    n = len(arr)  
    if n > k:  
        return True  
    possible = set()  
    for num in arr:  
        new = possible.copy()  
        new.add(num % k)  
        for rem in possible:  
            new.add((rem + num) % k)  
        possible = new
```

```
    if 0 in possible:
        return True

return False
```

Pattern Learned

Whenever you see:

Subset
+
Divisible by k

Immediately think:

Take / Not Take
↓
State = (index, sum)
↓
Can sum be replaced by $\text{sum} \% k$?
↓
Remainder has only k states
↓
Modulo DP

Real World DP Pattern

Actual value is huge
↓
Question only cares about remainder/property
↓
Compress state
↓
Store only important information

This is called:

Modulo State Compression

Similar Problems

1. Continuous Subarray Sum

(Leetcode 523)

2. Subarrays Divisible by K

(Leetcode 974)

3. Count of subsets whose sum is divisible by K

4. Partition array into subsets with equal sum

5. Target Sum

(Leetcode 494)

6. Partition to K Equal Sum Subsets

(Leetcode 698)

7. Closest Subsequence Sum

(Leetcode 1755)

8. Count subsets with given sum

9. Partition Equal Subset Sum

(Leetcode 416)

10. Perfect Sum Problem

Identification Template

Whenever the question contains:

```
sum divisible by k  
multiple of k  
modulo  
remainder
```

Ask yourself:

Do I really need the actual sum,
or only $\text{sum} \% k$?

If the answer is:

Only $\text{sum} \% k$

then:

Modulo DP / State Compression

should immediately come to your mind.